

MiniGameParty

VR Technical Document

OpenXR 기반 VR 인터랙션, 커스텀 이동, Behavior Tree AI

VR Interaction

Custom Movement

Shark AI

Baseball Physics

World UI

AUTHOR

유재우

Unreal Gameplay Programmer

MiniGameParty / VR Mini-game

CORE SYSTEMS

- OpenXR / Enhanced Input 기반 HMD 및 모션 컨트롤러 입력
- 손 포즈 블렌딩, 물리 오브젝트 Grab, 3D World UI
- 잠수함 전용 MovementComponent와 충돌 반사 처리
- AI Perception / Behavior Tree 기반 상어 전투 AI
- 스윙 속도 기반 야구 타격력 및 비거리 계산

MiniGameParty

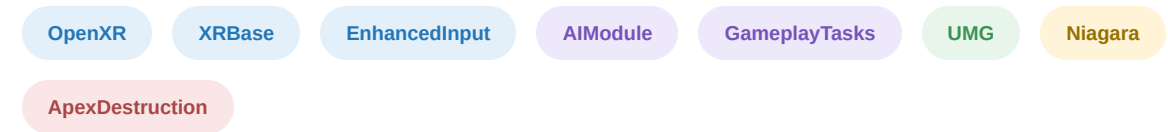
OpenXR 기반 공통 VR 인터랙션 위에 잠수함 조종 게임과 야구 타격 게임을 구성했습니다. 입력, 손 표현, Grab, 월드 UI, 커스텀 이동, AI, 물리 타격을 C++ 중심으로 구현했습니다.

Project	MiniGameParty
Genre	VR Mini-game Party
Engine	Unreal Engine 5.6
Language	C++ / Blueprint (애셋·UI 바인딩)
Platform	PC VR / Android (OpenXR)
Device	Meta Quest 2
Development	1인 개발

Implementation Scope

- VR Foundation - HMD, MotionController, 손 메시 및 포즈
- Interaction - GrabComponent, Sphere Trace, World UI
- Submarine - 기어, 관성 이동, 조향, 충돌 반사, 사격
- Shark AI - Perception, Blackboard, Strafe, Attack
- Baseball - 스윙 힘 계산, 임펄스, 착지 거리 측정

Engine Modules



VR 입력, AI 의사결정, 월드 공간 UI, 이펙트와 물리 상호작용을 기능 단위로 분리해 사용합니다.

Experience Structure

공통 VR 기반 위에 두 개의 미니게임 경험을 분리해 구성했습니다.

Playable Experiences



Underwater Combat

잠수함 조종석에서 기어와 조향을 제어하고 상어 AI와 전투합니다. 수중 관성 이동과 충돌 반사는 전용 MovementComponent가 담당합니다.



Baseball Simulation

모션 컨트롤러로 배트를 휘둘러 공을 타격합니다. 배트 이동량으로 힘을 계산하고 착지 거리까지 측정합니다.

Underwater Combat

- VR 탑승 및 조종석 UI
- 기어 3단계 속도 제어
- 입사각 기반 충돌 반사
- 상어 선회 / 돌진 AI
- 피격 균열과 카메라 셰이크

Baseball Simulation

- 배트 프레임 이동량으로 속도·가속도 계산
- $F = m a$ 기반 타격 힘 산출
- 공에 Physics / AddImpulse 적용
- 착지 지점까지 거리 계산 및 UI 표시

Main Menu -> Underwater / BaseballWidgetSwitcher + Delegate + OpenLevel

System Architecture

VR 입력, 상호작용, 시뮬레이션, AI를 계층별로 분리한 전체 구조입니다.

AVRCharacter

HMD / Hands

ASubMarine

Vehicle Pawn

AShark

Enemy Pawn

ABat / ABall

Baseball Actors

MotionController L/R

Tracking

UHandGraph

Input -> Pose

UGrabComponent

Grabbable

UWidgetInteraction

World UI

USubMarineMovement

Move / Collision

BT Task / Service

AI Decision

Input Layer

- Enhanced Input Mapping Context
- InputDataConfig로 IA 경로 중앙화
- Started / Triggered / Completed 바인딩

Interaction Layer

- 손 포즈 알파 및 미러링
- Sphere Trace 기반 Grab
- WidgetInteraction 기반 3D UI

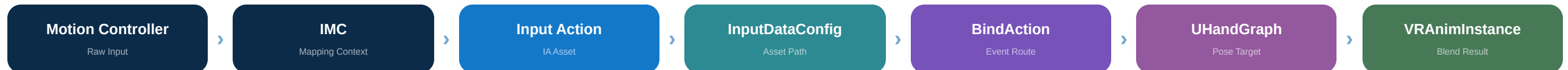
Simulation Layer

- 잠수함 관성 이동과 속도 제한
- 충돌 반사 및 조향 연동
- 배트 힘, 공 임펄스, 비거리

AI Layer

- AI Perception / Team Attitude
- Blackboard 상태 동기화
- Strafe / Attack Task와 Service

Input Pipeline



우선순위가 다른 Mapping Context를 분리하고, Input Action 애셋 참조는 ***InputDataConfig**에서 한곳에 관리합니다. 동일 입력값은 캐릭터 동작과 손가락 PoseAlpha에 각각 전달됩니다.

Pose Blending

- Grab / Point / Index / Thumb 입력을 0~1 알파값으로 전달
- AnimBlueprint에서 포즈별 블렌딩
- FInterpTo로 현재 알파를 목표 알파까지 보간

Left / Right Reuse

- 좌우 손이 동일한 ABP_Hand와 로직 공유
- bMirror 플래그로 왼손 포즈 반전
- 좌우 래퍼 함수는 Controller 포인터만 분기

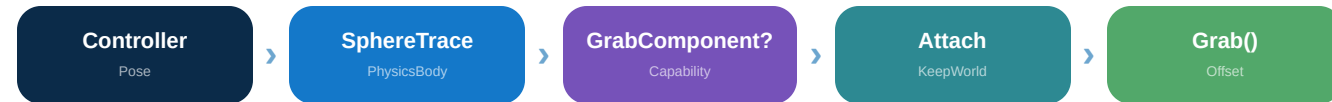
HandGraph -> VRAnimInstance

```
ActionValue -> PoseAlphaGrab  
CurrentAlpha = FInterpTo(  
    CurrentAlpha, TargetAlpha,  
    DeltaSeconds, 13.0f);
```

Grab Interaction & World UI

컴포넌트 보유 여부로 잡기 대상을 판정하고, 별도 채널로 3D UI를 조작합니다.

Physics Grab Flow



SphereTrace 결과가 **UGrabComponent**를 가진 PhysicsBody일 때만 잡기 대상으로 인정합니다. 액터 타입이 아닌 기능 컴포넌트로 판정해 새 오브젝트도 같은 흐름에 연결합니다.

World UI Interaction



전용 **WorldUI** 채널로 게임 충돌과 UI 충돌을 분리하고, 검지 입력을 Pointer Key Press / Release로 변환해 UMG의 Hover와 Click을 그대로 재사용합니다.

NORMAL



선택되지 않은 기본 상태

HOVER



VR 포인터가 버튼 위에 위치한 상태

CLICK



클릭 후 선택 테두리가 적용된 상태

Grab Capability

PhysicsBody + UGrabComponent 조합으로 대상 판정

Trace Separation

WorldUI 전용 채널로 게임 충돌과 UI 충돌 분리

UMG Reuse

Pointer Key 변환으로 기존 Hover / Click 이벤트 재사용

06

MINIGAME PARTY

Menu & Level Selection

WidgetSwitcher와 델리게이트 기반 버튼 위젯으로 화면 전환과 레벨 선택을 처리합니다.

WidgetSwitcher



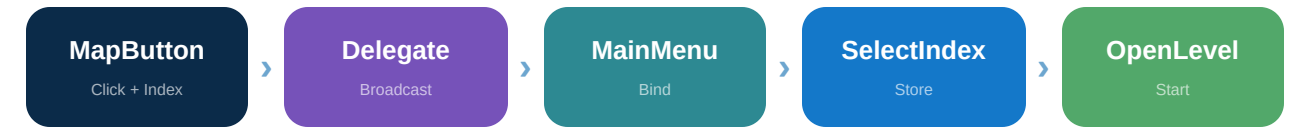
Start Screen



Map Select

하나의 UMG 위젯 안에 시작 화면과 맵 선택 화면을 두고, WidgetSwitcher의 Active Widget Index를 변경해 전환합니다. 맵 버튼의 Normal / Hover / Click 상태는 버튼 위젯 내부에 캡슐화했습니다.

Selection Flow



각 버튼은 자신의 Index를 델리게이트로 전달합니다. 메인 메뉴는 선택 Index를 저장하고 버튼 배열을 순회해 선택 항목만 On, 나머지는 Off로 갱신합니다.

Delegate + Index

버튼 위젯이 메인 메뉴를 직접 참조하지 않고 선택 이벤트만 브로드캐스트합니다.

Level Data Lookup

Start 클릭 시 SelectIndex로 레벨 배열 또는 DataTable을 조회하고 OpenLevel을 호출합니다.

Button Widget -> Delegate(Index) -> Main Menu -> OpenLevel

Submarine System Architecture

탐승, 조향, UI, 이동, 충돌, 사격을 역할별 객체로 분리했습니다.

VR Cockpit



플레이어가 잠수함에 탑승하면 모션 컨트롤러 입력과 조종석 UI가 ASubMarine의 조향, 기어, 발사 동작으로 라우팅됩니다.

Runtime Responsibilities

AVRCharacter

탐승 / 하차, VR 입력 라우팅

ASubMarine

조향, 발사, 피격, UI 이벤트

USubMarineMovement

Velocity, Gear, SafeMove, Collision

UI_SubMarine

기어 선택, 발사, 상태 표시

Control & Combat Flow

VR Input

Hands / UI

ASubMarine

Control

Movement

SafeMove

World

Hit / Move

HOSTILE RELATIONSHIP

Shark AI Controller는 팀 판정을 통해 ASubMarine만 Hostile 대상으로 인식하고 양쪽 Damage 흐름을 연결합니다.

Custom Movement & Gear

수중 관성 이동을 위해 UFloatingPawnMovement를 확장하고 기어 기반 속도 제한을 적용했습니다.

Gear Table

GEAR 1 500 Max Speed	GEAR 2 1000 Max Speed	GEAR 3 1500 Max Speed
-----------------------------------	------------------------------------	------------------------------------

ESubmarineGear::Max를 배열 크기로 사용하고 현재 기어에 대응하는 MaxSpeed를 선택합니다.

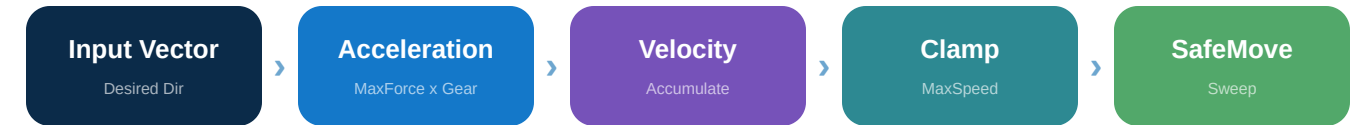
Velocity Update

```
Velocity += Input * MaxForce * (Gear + 1)
```

```
if Speed > MaxSpeed: Velocity = Normal * MaxSpeed
```

입력 방향에 가속력을 누적해 관성을 만들고, 기어별 최고 속도를 초과하면 방향을 유지한 채 크기만 제한합니다.

Movement Pipeline



```
USubmarineMovementComponent  
  
void InputVector(FVector Input)  
{  
    Velocity += Input * MaxForce * (Gear + 1);  
    if (Velocity.Size() > MaxSpeed)  
        Velocity = Velocity.GetSafeNormal() * MaxSpeed;  
}
```

WHY A CUSTOM MOVEMENTCOMPONENT?

수중 관성, 기어 제한, Sweep 이동, 충돌 반사와 입력 차단을 하나의 이동 책임 안에서 일관되게 처리하기 위해 전용 컴포넌트로 분리했습니다.

Collision Response

Impact Normal과 이동 방향의 내적으로 정면 충돌과 스침을 구분합니다.

```
d = Dot(ImpactNormal, Velocity.GetSafeNormal())
```

```
d < -0.5 -> Frontal Impact  
else -> Glancing / Edge
```

```
Reflection = GetReflectionVector(Velocity, Hit.Normal)
```

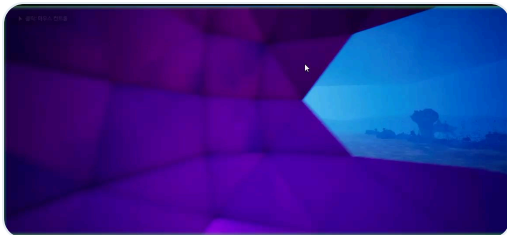
Collision Cases



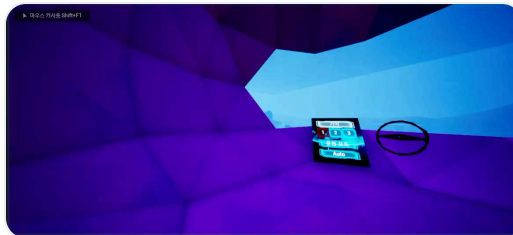
FRONT



LOW ANGLE



SIDE



CORNER

Frontal Impact

입사 방향이 면의 정면에 가까우면 반사 속도를 더 크게 감쇠하고 0에 가까워지는 값을 보정합니다.

Glancing / Edge

비스듬한 충돌은 반사 방향을 유지하면서 속력만 줄여 벽을 따라 미끄러지는 반응을 만듭니다.

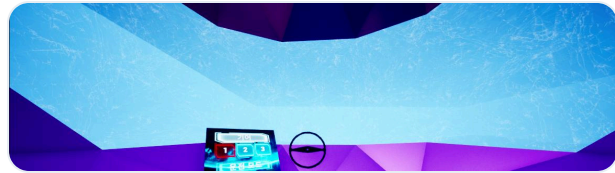
Stuck Prevention

충돌 직후 같은 방향 입력이 계속 누적되지 않도록 1초간 입력을 차단한 뒤 자동 해제합니다.

Steering & Damage Feedback

물리 조향 입력을 선체 회전으로 변환하고, 피격 상태를 재질과 카메라로 전달합니다.

Progressive Window Damage



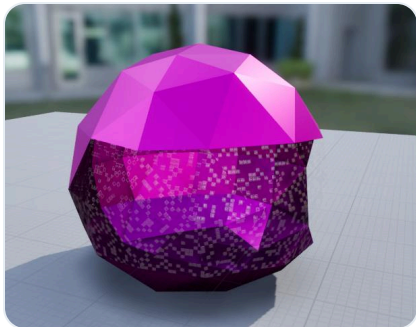
Damage Stage 1



Damage Stage 2

TakeDamage에서 Dynamic Material Instance의 Window 파라미터를 누적 변경해 조종석 유리 균열 강도를 단계적으로 높입니다.

Submarine Presentation



Steering Wheel -> Hull Yaw

Wheel Pitch

-60 ~ +60

Normalize

/ -60

Add Yaw

Hull Rotation

- 운전 모드에서만 Steering 입력 처리
- 핸들 Pitch를 잠수함 Yaw 증가량으로 변환
- 손을 놓으면 RInterpTo로 중앙 복귀
- 피격 시 Material Parameter와 Camera Shake 동시 실행

Interaction Feedback

```
YawAngle = SteeringWheel.Pitch;
AddYaw = YawAngle / -60.0f;
AddActorWorldRotation(Yaw = AddYaw);
```

TakeDamage -> Window Param -> Camera Shake

Shark AI Decision System

AI Perception, Blackboard, Behavior Tree Task/Service로 선회와 돌진을 제어합니다.

AI Pipeline



Shark Enemy



타깃을 중심으로 원형 궤도를 계산해 선회하고 랜덤 대기 후 돌진합니다.

PERCEPTION & TEAM

Sight 반경 3000, 시야각 360°. ASubMarine만 Hostile 판정.

STRAFE

sin / cos로 원형 목표 위치 계산. 1~5초 후 Attack 전환.

ATTACK END

distance ≥ 2000 이고 dot < -0.7 이면 빛나감으로 판단해 종료.

PROCEDURAL SWIM

Timeline + Curve로 몸통 Yaw를 흔들고 완료 콜백으로 반복.

Baseball Hit Simulation

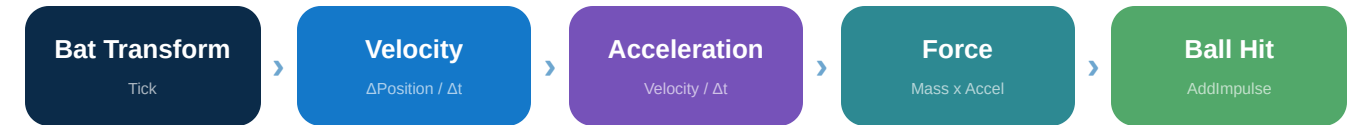
배트의 실제 프레임 이동량을 힘으로 변환하고 공의 착지 거리까지 계산합니다.

VR Batter View



모션 컨트롤러에 부착된 배트의 위치 변화를 매 프레임 추적하여 스윙 세기를 계산합니다.

Hit Processing Flow



Bat -> Ball

```
Velocity    = (Current - Previous) / DeltaTime;  
Acceleration = Velocity / DeltaTime;  
Force       = Mass * Acceleration;  
Sphere.AddImpulse(Force * SpeedScale);
```

F = m a

스윙이 빠를수록 큰 타격력

1~3x Pitch

투구 속도 랜덤화

SpawnActorDeferred

스폰 전 Projectile Speed 설정

cm / 100

착지 거리 m 단위 변환

Reusable Design & Troubleshooting

수정 지점을 줄이고 VR 특화 문제를 해결한 설계 포인트를 정리합니다.

Reusable Design

Component-based

Grab, Hand, Movement 기능을 독립 컴포넌트로 분리

Data-driven Input

Input Action 경로를 Config 클래스에서 중앙 관리

Left / Right Wrapper

공통 구현에 Controller 포인터만 전달해 중복 제거

Name Constants

Collision Preset / Channel 문자열을 상수 테이블로 관리

System Boundaries

Input

Enhanced Input / Config

Presentation

Hand Anim / Dynamic Material

Interaction

Grab / WidgetInteraction

Simulation

Movement / Physics / Distance

AI

Perception / Blackboard / BT

Content

Submarine / Baseball Actors

기능 책임을 경계별로 분리하여 손 표현, 이동 규칙, AI 로직, 미니게임 콘텐츠가 서로 직접 의존하지 않도록 구성했습니다.

Troubleshooting

Problem	Cause	Solution
잠수함이 벽에서 떨림 / 끼임	충돌 후 같은 방향 입력이 계속 누적	BlockingDelayInputTime으로 입력 일시 차단
모든 충돌이 같은 세기로 반사	입사각이 반사 속도에 반영되지 않음	ImpactNormal과 Velocity 내적으로 정면 / 스침 분리
상어가 타격을 지나쳐도 돌진 유지	공격 실패 판정이 없음	Service에서 거리 + 내적으로 EndAttack 호출
손 포즈가 즉시 바뀌어 딱딱함	PoseAlpha를 직접 대입	FInterpTo로 현재 알파를 목표값까지 보간
약한 스윙과 강한 스윙 결과가 동일	충돌 이벤트만으로 타격 처리	배트 프레임 이동량으로 Force 계산 후 AddImpulse

Technical Summary

MiniGameParty에서 구현한 핵심 기술을 시스템 단위로 요약합니다.

OpenXR Input

HMD와 모션 컨트롤러 입력을 Enhanced Input으로 추상화

Hand Pose

PoseAlpha와 FInterpTo로 손가락 포즈를 연속 블렌딩

Grab / World UI

컴포넌트 기반 Grab과 전용 채널의 3D UMG 클릭

Custom Movement

기어, 관성, 속도 제한, Sweep 이동을 전용 컴포넌트로 처리

Collision Reflection

내적값으로 충돌 방향을 분류하고 반사 속도를 차등 적용

Shark AI

Perception, Blackboard, BT Task/Service로 선회와 돌진 제어

Baseball Physics

배트 이동량에서 Force를 계산하고 Ball에 Impulse 적용

Data-driven Config

Input Action과 충돌 이름의 수정 지점을 중앙화

SYSTEM SUMMARY

- 컨트롤러 입력 -> 손 애니메이션 -> 물리 상호작용으로 이어지는 VR 파이프라인을 C++로 구성했습니다.
- 잠수함 이동과 상어 AI처럼 프로젝트 고유 규칙은 엔진 기반 클래스를 상속해 별도 시스템으로 확장했습니다.
- 두 미니게임은 공통 VR Character와 Interaction 기반을 공유하고 콘텐츠별 시뮬레이션만 분리합니다.